

Queue Management System

An Odoo 17 Module — Architecture & Line-by-Line Code Walkthrough

QMS project — learning guide

2026-06-23

Contents

1	Introduction	3
2	The big picture: how an Odoo module is layered	4
3	The module descriptor: <code>__manifest__.py</code>	5
3.1	Key lessons	6
4	Python package wiring: the <code>__init__.py</code> files	7
4.1	Key lesson	7
5	The ORM layer: <code>models/</code>	8
5.1	<code>qms_service_plan.py</code> — the simplest model	8
5.1.1	Lessons	9
5.2	<code>qms_service.py</code> — relations and a computed field	9
5.2.1	Lessons	10
5.3	<code>qms_queue.py</code> — <code>_rec_name</code> , SQL constraint, <code>create()</code> override	10
5.3.1	Lessons	11
5.4	<code>qms_ticket.py</code> — the richest model	11
5.4.1	Lessons	12
6	Security: <code>security/ir.model.access.csv</code>	13
6.1	Lessons	13
7	The view layer: <code>views/*.xml</code>	14
7.1	The standard pattern (<code>qms_queue_views.xml</code>)	14
7.1.1	Lessons — the core Odoo pattern	16
7.2	Extras worth knowing (<code>qms_ticket_views.xml</code>)	16
7.3	Widgets (<code>qms_service_views.xml</code>)	16
7.4	Embedded one2many (<code>qms_service_plan_views.xml</code>)	17
7.5	Actions and menus (<code>qms_menus.xml</code>)	17
7.5.1	Lessons	17
8	Website templates: server-side QWeb (<code>qms_public_templates.xml</code>)	18
8.1	Lessons	19
9	The controller layer: <code>controllers/main.py</code>	20
9.1	Lessons	21

10 The frontend layer: <code>static/src/</code> (OWL / JS / CSS)	22
10.1 <code>qms_console.js</code> — the OWL component	22
10.1.1 Lessons	23
10.2 <code>qms_console.xml</code> — the OWL template	23
10.2.1 Lessons	24
10.3 <code>qms_console.css</code> — the collapse styling	24
11 The end-to-end flow	26
12 The data model and concept	27
13 Development and deployment workflow	28
13.1 Common commands	28
14 Glossary / cheat-sheet	29
15 Where this fits in the learning roadmap	30

Chapter 1

Introduction

This book explains the **Queue Management System (QMS)** Odoo 17 module file by file, line by line. It is written as a learning guide: the QMS project is deliberately built as a *tour of Odoo's UI rendering techniques*, so reading its code teaches how the whole Odoo architecture fits together.

By the end you should understand:

- How an Odoo module is structured and loaded.
- The **ORM layer** (models, fields, relations, computed fields, overrides).
- **Security** (access rights).
- The **declarative view layer** (list/form/search views, actions, menus).
- **Website templates** (server-side QWeb) and **controllers** (HTTP).
- The **frontend layer** (OWL components, JS services, QWeb, CSS).
- How a request flows end to end through these layers.

The module lives at `projectspace/myodoo/addons/qms`.

Chapter 2

The big picture: how an Odoo module is layered

An Odoo module is organised into layers. Each file in QMS belongs to one of them:

<code>__manifest__.py</code>	-> the module descriptor (Odoo reads this first)
<code>__init__.py</code>	-> Python package wiring
<code>models/</code>	-> ORM layer (Python): business data + logic
<code>security/</code>	-> access rights (CSV)
<code>views/</code>	-> declarative UI (XML): backend screens, menus, web pages
<code>controllers/</code>	-> HTTP layer (Python): public website routes
<code>static/src/</code>	-> frontend assets (JS / OWL / CSS): custom client UI
<code>docs/</code>	-> this book

Odoo is **MVC-like**:

- **Model** = the ORM (Python classes mapped to PostgreSQL tables).
- **View** = XML view definitions (rendered by a generic engine) and OWL components (custom JS UI).
- **Controller** = HTTP routes for the public website.

Crucially, Odoo has **two UI runtimes**:

1. The **backend web client** — a JavaScript single-page app (OWL) used by logged-in staff. Standard screens are *declarative* (XML), custom ones are *client actions* (OWL).
2. The **website / frontend** — server-rendered HTML (Python QWeb) for the public.

QMS uses both, which is exactly why it is a good teaching example.

Chapter 3

The module descriptor: __manifest__.py

This is the first file Odoo reads. It declares the module's identity, dependencies, and the files to load.

```
{
    "name": "Queue Management System",          # human name in the Apps list
    "version": "17.0.1.0.0",                   # <odoo_ver>.<module_ver>; drives
    ↪ upgrades
    "summary": "Queue Management System - a learning tour of Odoo UI rendering",
    "category": "Services",                    # grouping in Apps
    "author": "Norun Nabi",
    "license": "LGPL-3",
    "depends": ["base", "website"],             # modules that MUST load before this
    ↪ one
    "data": [                                  # XML/CSV loaded at install/upgrade, IN
    ↪ ORDER
        "security/ir.model.access.csv",
        "views/qms_service_plan_views.xml",
        "views/qms_service_views.xml",
        "views/qms_queue_views.xml",
        "views/qms_ticket_views.xml",
        "views/qms_menus.xml",
        "views/qms_public_templates.xml",
    ],
    "assets": {                                # frontend bundles (JS/CSS/QWeb)
        "web.assets_backend": [                # loaded into the backend web client
            "qms/static/src/css/qms_console.css",
            "qms/static/src/js/qms_console.js",
            "qms/static/src/xml/qms_console.xml",
        ],
    },
    "application": True,                       # shows as a top-level App (icon in
    ↪ grid)
    "installable": True,
```

```
}
```

3.1 Key lessons

- **depends** — Odoo builds a dependency graph and loads modules in order. We need **base** (core) and **website** (for the public page, **website.layout**, and **website.menu**).
- **data is order-sensitive.** Security first (so models are accessible), then views that *define* actions, then **qms_menus.xml** which *references* those actions, then website templates. Loading a menu before its action causes an install error.
- **assets is separate from data.** **data** creates server-side records; **assets** are browser files bundled into **web.assets_backend** (the backend JS/CSS bundle).
- **version** — bump it and Odoo runs the module's upgrade on the next **-u**.

Chapter 4

Python package wiring: the `__init__.py` files

Odoo imports your module as a Python package, so every Python folder needs an `__init__.py`.

Root `__init__.py`:

```
from . import models          # import the models sub-package
from . import controllers     # import the controllers sub-package
```

`models/__init__.py`:

```
from . import qms_service_plan # plan before service
from . import qms_service      # (service.plan_id references the plan model)
from . import qms_queue
from . import qms_ticket
```

`controllers/__init__.py`:

```
from . import main
```

4.1 Key lesson

Importing a model file is what *registers* the model class with Odoo's registry. If you create a model file but forget to import it here, **Odoo never sees it** — no table is created. This is the single most common beginner bug.

Chapter 5

The ORM layer: models/

This is the heart of Odoo. Each Python class is one database table plus a rich API. We cover all four QMS models, from simplest to richest.

5.1 qms_service_plan.py — the simplest model

```
from odoo import fields, models          # fields = column types, models = base
→ classes

class QmsServicePlan(models.Model):      # models.Model = a persistent DB-backed
→ model
    _name = "qms.service.plan"           # technical id -> table name
→ qms_service_plan
    _description = "QMS Service Plan"     # required; human label
    _order = "name"                      # default sort

    name = fields.Char(required=True)      # VARCHAR column, NOT NULL
    code = fields.Char(help="Short product code.") # help = tooltip
    description = fields.Text()            # multi-line TEXT column
    price = fields.Float(string="Price")    # numeric; string= overrides the UI
→ label
    active = fields.Boolean(default=True)   # special name 'active' = archive
→ support

    service_ids = fields.One2many(         # virtual field: the "many" side of a
→ relation
        "qms.service",                   # the related model
        "plan_id",                       # the Many2one field on qms.service pointing back here
        string="Subscriptions",
    )
```

5.1.1 Lessons

- `_name` makes Odoo auto-create the table `qms_service_plan` and an ORM proxy. Dots become underscores.
- Fields are **declarative**: you describe columns; Odoo generates the SQL and the UI metadata.
- `active` is **magic**: a Boolean named `active` gives you archive/unarchive for free (archived rows are hidden from searches).
- One2many is **not a column** — it is computed from the reverse Many2one. `service_ids` reads “all `qms.service` rows whose `plan_id` is me.”

5.2 `qms_service.py` — relations and a computed field

```
from odoo import api, fields, models      # api = decorators (depends,
↳ model_create_multi)

class QmsService(models.Model):
    _name = "qms.service"
    _description = "QMS Service"
    _order = "sequence, name"              # sort by sequence first, then name

    name = fields.Char(required=True)
    code = fields.Char(help="Short prefix, e.g. 'B' for Billing.")

    plan_id = fields.Many2one(              # FK -> stored as integer column
↳ plan_id
        "qms.service.plan", string="Service Plan")

    customer_company_id = fields.Many2one(
        "res.partner",                    # reuse Odoo's built-in partner model
        string="Buyer Company",
        domain=[("is_company", "=", True)], # UI filter: only company partners
        help="The company that bought this service from the QMS provider.")

    state = fields.Selection(              # dropdown stored as VARCHAR key
        selection=[("draft", "Draft"), ("running", "Running"),
↳ ("expired", "Expired")],
        default="draft", required=True)

    date_start = fields.Date(string="Start Date")
    date_end = fields.Date(string="End Date")
    sequence = fields.Integer(default=10)  # manual ordering
    active = fields.Boolean(default=True)
    color = fields.Integer(string="Color Index")
    description = fields.Text()

    queue_ids = fields.One2many("qms.queue", "service_id", string="Queues")
    queue_count = fields.Integer(          # computed, not stored
        string="# Queues", compute="_compute_queue_count")
```

```

@api.depends("queue_ids")                # recompute when queue_ids changes
def _compute_queue_count(self):
    grouped = self.env["qms.queue"].read_group(    # one SQL GROUP BY, not N
↪ queries
        domain=[("service_id", "in", self.ids)],
        fields=["service_id"], groupby=["service_id"])
    counts = {g["service_id"][0]: g["service_id_count"] for g in grouped}
    for service in self:                    # self is a recordset (can be many
↪ records)
        service.queue_count = counts.get(service.id, 0)

```

5.2.1 Lessons

- Many2one is a foreign key. `plan_id` is an integer column; in Python `service.plan_id` returns the *related record* (lazy-loaded), not the id.
- `domain` on a relation field filters what the UI lets you pick (a UX filter).
- **Reusing `res.partner`** instead of a custom company/customer model is the Odoo way; you inherit mail, addresses, portal, and invoicing later.
- **Computed fields:** `compute=` names a method; `@api.depends(...)` tells Odoo *when* to recompute. Not stored by default -> recalculated on read.
- `self` is a **recordset** (a collection), so compute methods loop for `service in self`. ORM methods operate on sets, not single rows.
- `read_group` performs a SQL GROUP BY in one query, avoiding the N+1 problem of `len(service.queue_ids)` per record.

5.3 qms_queue.py — `_rec_name`, SQL constraint, `create()` override

```

class QmsQueue(models.Model):
    _name = "qms.queue"
    _description = "QMS Queue"
    _rec_name = "q_name"                # which field is the "display name" (default
↪ 'name')
    _order = "quid"

    quid = fields.Integer(string="Queue ID", required=True, copy=False, index=True,
        help="Unique id for the object.")
    # copy=False -> not duplicated on "Duplicate"; index=True -> DB index
    q_name = fields.Char(string="Name", required=True)
    q_description = fields.Char(string="Description")
    created_at = fields.Datetime(default=fields.Datetime.now, readonly=True)
    updated_at = fields.Datetime(readonly=True)

    service_id = fields.Many2one("qms.service", required=True, ondelete="cascade")
    # ondelete="cascade" -> delete the service, its queues go too

    ticket_ids = fields.One2many("qms.ticket", "quid", string="Tickets")

```

```

ticket_count = fields.Integer(compute="_compute_ticket_count")

_sql_constraints = [
    ↪ Python)
    ("quid_unique", "unique(quid)", "Queue ID (quid) must be unique."),
]

@api.model_create_multi
    ↪ vals dicts
def create(self, vals_list):
    now = fields.Datetime.now()
    last = self.search([], order="quid desc", limit=1)    # highest existing
    ↪ quid
    next_quid = (last.quid + 1) if last else 1
    for vals in vals_list:
        if not vals.get("quid"):
            vals["quid"] = next_quid    # auto-number
            next_quid += 1
        vals.setdefault("created_at", now)
        vals.setdefault("updated_at", now)
    return super().create(vals_list)    # ALWAYS call super() to do the real
    ↪ insert

def write(self, vals):
    vals.setdefault("updated_at", fields.Datetime.now())
    return super().write(vals)

```

5.3.1 Lessons

- `_rec_name = "q_name"` — because our spec did not use `name`, we tell Odoo which field to show in dropdowns and breadcrumbs.
- `_sql_constraints` is enforced by **PostgreSQL itself** (a real `UNIQUE` index), stronger than a Python check.
- **Overriding `create()`/`write()`** is the canonical place to inject logic (auto-numbering, timestamps). Golden rule: **always return `super().create(...)`** or the record is never inserted.
- `@api.model_create_multi` is the Odoo 17 convention — `create` receives a *list* of dicts for batch inserts.
- `ondelete` defines referential behaviour (`cascade` / `restrict` / `set null`).

5.4 qms_ticket.py — the richest model

```

class QmsTicket(models.Model):
    _name = "qms.ticket"
    _description = "QMS Ticket"
    _rec_name = "ticket_name"
    _order = "tid desc"    # newest first

```

```

tid = fields.Integer(string="Ticket ID", required=True, copy=False, index=True)
quid = fields.Many2one("qms.queue", string="Queue",
                      required=True, ondelete="cascade", index=True)
ticket_name = fields.Char(string="Name")
ticket_description = fields.Char(string="Description")
customer_id = fields.Many2one("res.partner", string="Customer")
created_at = fields.Datetime(default=fields.Datetime.now, readonly=True)
updated_at = fields.Datetime(readonly=True)

state = fields.Selection([("waiting", "Waiting"), ("called", "Called"),
                          ("serving", "Serving"), ("done", "Done"), ("cancelled", "Cancelled")],
                        default="waiting", required=True, index=True)
position = fields.Integer(compute="_compute_position") # live, not stored
notified = fields.Boolean(default=False)

_sql_constraints = [("tid_unique", "unique(tid)", "Ticket ID (tid) must be
↪ unique.")]

@api.depends("quid", "state", "created_at")
def _compute_position(self):
    for ticket in self:
        if ticket.state == "waiting" and ticket.quid:
            ahead = self.search_count([ # COUNT of tickets ahead in
↪ line
                ("quid", "=", ticket.quid.id),
                ("state", "=", "waiting"),
                ("created_at", "<", ticket.created_at or
↪ fields.Datetime.now()),
            ])
            ticket.position = ahead + 1 # 1 = next up
        else:
            ticket.position = 0
    # create()/write() follow the same auto-number + timestamp pattern as the
↪ queue

```

5.4.1 Lessons

- quid here is a **Many2one to the queue** (the spec named the FK “quid”). So ticket.quid is the queue *record*; ticket.quid.q_name walks the relation.
- position is a **non-stored computed field** that queries other records (“how many waiting tickets are ahead of me”). This is the seed of the “notify before your turn” concept.
- search_count / search are the ORM’s query API: you pass a **domain** (a list of (field, operator, value) tuples) instead of SQL.

Chapter 6

Security:

`security/ir.model.access.csv`

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
|
↪ access_qms_service_plan_user,qms.service.plan.user,model_qms_service_plan,base.group_user,1
access_qms_service_user,qms.service.user,model_qms_service,base.group_user,1,1,1,1
access_qms_queue_user,qms.queue.user,model_qms_queue,base.group_user,1,1,1,1
access_qms_ticket_user,qms.ticket.user,model_qms_ticket,base.group_user,1,1,1,1
```

6.1 Lessons

- This CSV loads into the `ir.model.access` table. Each row says “group X has these CRUD permissions on model Y.”
- `model_id:id` uses the magic xmlid `model_<name_with_underscores>` that Odoo auto-creates for every model.
- `base.group_user` is the “Internal User” group. The four 1s are read / write / create / unlink.
- **Critical rule:** a model with *no* access line is invisible/unusable in the UI. This is why the public controller uses `sudo()` — anonymous visitors are not in `group_user`.

Chapter 7

The view layer: views/*.xml

Views are **data records** (rows in `ir.ui.view`, `ir.actions.*`, `ir.ui.menu`) written in XML. Odoo's generic renderer turns them into UI.

7.1 The standard pattern (`qms_queue_views.xml`)

```
<odoo>
  <record id="view_qms_queue_list" model="ir.ui.view">      <!-- create an
→  ir.ui.view row -->
    <field name="name">qms.queue.list</field>                <!-- internal label -->
    <field name="model">qms.queue</field>                    <!-- which model it
→  renders -->
    <field name="arch" type="xml">                            <!-- the view definition
→  -->
    <tree>                                                    <!-- 'tree' = list view
→  -->
        <field name="quid"/>                                  <!-- a column -->
        <field name="q_name"/>
        <field name="service_id"/>
        <field name="ticket_count"/>
        <field name="created_at"/>
    </tree>
  </field>
</record>

  <record id="view_qms_queue_form" model="ir.ui.view">
    <field name="name">qms.queue.form</field>
    <field name="model">qms.queue</field>
    <field name="arch" type="xml">
      <form>
        <sheet>                                              <!-- the white "paper"
→  card -->
          <div class="oe_title">
```

```

        <label for="q_name"/>
        <h1><field name="q_name" placeholder="e.g. Counter
→ A"/></h1>

        </div>
        <group>                                <!-- 2-column responsive
→ layout -->

            <group>
                <field name="quid" readonly="1"/>
                <field name="service_id"/>
            </group>
            <group>
                <field name="created_at"/>
                <field name="updated_at"/>
            </group>
        </group>
        <field name="q_description"/>
    </sheet>
</form>
</field>
</record>

<record id="view_qms_queue_search" model="ir.ui.view">
    <field name="name">qms.queue.search</field>
    <field name="model">qms.queue</field>
    <field name="arch" type="xml">
        <search>
            <field name="q_name"/>                                <!-- searchable fields
→ -->

            <field name="quid"/>
            <field name="service_id"/>
            <group expand="0" string="Group By">
                <filter name="group_service" string="Service"
                    context="{ 'group_by': 'service_id' }"/> <!-- group-by
→ toggle -->

            </group>
        </search>
    </field>
</record>

<record id="action_qms_queue" model="ir.actions.act_window"> <!-- the "open
→ me" action -->
    <field name="name">Queues</field>
    <field name="res_model">qms.queue</field>
    <field name="view_mode">tree,form</field>                                <!-- list first, then
→ form -->
    <field name="search_view_id" ref="view_qms_queue_search"/>
</record>

```

</odoo>

7.1.1 Lessons — the core Odoo pattern

- A **view** (`ir.ui.view`) describes *how* to render a model; the `<arch>` is the layout.
- One model can have **many view types**: `tree` (list), `form`, `search`, `kanban`, `pivot`, `graph`, `calendar`, and more.
- An **action** (`ir.actions.act_window`) ties a model to its views and is what a menu or button opens. `view_mode="tree,form"` opens as a list, click a row to get the form.
- The **search view** powers the search box, filters, and group-by, declaratively.
- The renderer wires up save/discard, validation, pagination, and breadcrumbs for free. You wrote zero JavaScript. This is why Odoo can ship thousands of screens.

7.2 Extras worth knowing (`qms_ticket_views.xml`)

```
<tree decoration-muted="state in ('done','cancelled')"    <!-- grey out finished
→ rows -->
    decoration-bf="state == 'serving'"                  <!-- bold the serving row
    → -->
...
<header>
    <field name="state" widget="statusbar"                <!-- the pipeline at the
→ top -->
        statusbar_visible="waiting,called,serving,done"/>
</header>
...
<field name="context">{'search_default_waiting': 1}</field> <!-- auto-apply
→ 'Waiting' filter -->
```

- `decoration-*` colours rows by a condition on the record.
- `widget="statusbar"` renders a Selection as the clickable pipeline at the top of forms.
- `search_default_<filtername>` in an action's context pre-activates a filter when the screen opens.

7.3 Widgets (`qms_service_views.xml`)

```
<field name="sequence" widget="handle"/>    <!-- drag-to-reorder dots in the list
→ -->
<field name="color" widget="color_picker"/>    <!-- a colour swatch picker in the
→ form -->
<field name="state" widget="statusbar" statusbar_visible="draft,running,expired"/>
```

- **Widgets** control how a single field is rendered. Same data, different UI.

7.4 Embedded one2many (qms_service_plan_views.xml)

```
<notebook>                                <!-- tabbed area -->
  <page string="Subscriptions">
    <field name="service_ids">             <!-- show the One2many as a sub-list
    ↪ -->
      <tree>                                <!-- inline view, just for this embed
    ↪ -->
        <field name="name"/>
        <field name="customer_company_id"/>
        <field name="state"/>
      </tree>
    </field>
  </page>
</notebook>
```

- A One2many field can render an inline editable sub-table inside the parent form. This is how Odoo shows order lines and similar.

7.5 Actions and menus (qms_menus.xml)

```
<record id="action_qms_console" model="ir.actions.client"> <!-- a CLIENT action
    ↪ -->
  <field name="name">QMS Console</field>
  <field name="tag">qms_console</field>           <!-- key linking to the JS
    ↪ component -->
</record>

<menuitem id="menu_qms_root" name="Queue Management System"
  groups="base.group_user" sequence="10"/>       <!-- top-level App menu
    ↪ -->
<menuitem id="menu_qms_home" name="Home" parent="menu_qms_root"
  action="action_qms_console" sequence="10"/>     <!-- child -> opens
    ↪ console -->
```

7.5.1 Lessons

- `<menuitem>` is sugar for creating `ir.ui.menu` rows. A top-level menu with no action is an App; children point to actions.
- `groups=` restricts visibility (security at the menu level).
- Two **action types** appear here: `act_window` (opens model views) and `ir.actions.client` (opens a custom JS screen).
- We keep only one menu (Home -> Console) so the top bar stays clean; the `act_window` actions still exist and are opened from the console sidebar.

Chapter 8

Website templates: server-side QWeb (qms_public_templates.xml)

This is the **website runtime**: Python QWeb rendered to HTML on the server, for the public.

```
<template id="qms_book_form" name="Book a Queue Ticket">    <!-- a QWeb template
→ -->
    <t t-call="website.layout">                                <!-- wrap in site
→ header/footer -->
        <div class="container my-5" style="max-width: 640px;">
            <h1 class="mb-4">Book a ticket</h1>
            <t t-if="error">                                    <!-- conditional
→ rendering -->
                <div class="alert alert-danger"><span t-esc="error"/></div>
            </t>
            <form action="/qms/book/submit" method="post">
                <input type="hidden" name="csrf_token"
                    t-att-value="request.csrf_token()"/> <!-- t-att = dynamic
→ attribute -->
                <select id="quid" name="quid" required="required">
                    <option value="">- Choose a queue -</option>
                    <t t-foreach="queues" t-as="queue">          <!-- loop over server
→ data -->
                        <option t-att-value="queue.id">
                            <t t-esc="queue.service_id.name"/> - <t
→ t-esc="queue.q_name"/>
                                </option>
                        </t>
                    </select>
                </form>
            </div>
        </t>
    </template>
```

```

<record id="menu_qms_book_website" model="website.menu">    <!-- link in the PUBLIC
↳  nav -->
    <field name="name">Book a Ticket</field>
    <field name="url">/qms/book</field>
    <field name="parent_id" ref="website.main_menu"/>
</record>

```

8.1 Lessons

- **QWeb** is Odoo's templating language. Server-side QWeb renders final HTML on the server.
- Directives: **t-if** (condition), **t-foreach/t-as** (loop), **t-esc** (escaped output), **t-att-X** (dynamic attribute), **t-call** (include another template — here the site chrome).
- `request.csrf_token()` is required on `type="http"` POST forms.
- `website.menu` adds the link to the public site nav (different from the backend `ir.ui.menu`).
- Contrast with the backend views: those are declarative model views rendered by JS; this is hand-written HTML rendered by Python. Two different runtimes.

Chapter 9

The controller layer: controllers/main.py

Controllers handle raw HTTP — they are how the website talks to the ORM.

```
from odoo import http
from odoo.http import request                                # per-request object (env, params,
↳ session)

class QmsPublicController(http.Controller):                  # subclass http.Controller
    @http.route("/qms/book", type="http", auth="public", website=True,
↳ methods=["GET"])
    def qms_book_form(self, **kw):
        queues = request.env["qms.queue"].sudo().search([], order="service_id,
↳ quid")
        return request.render("qms.qms_book_form", {"queues": queues,
                                                    "error": kw.get("error")})

    @http.route("/qms/book/submit", type="http", auth="public", website=True,
                methods=["POST"])
    def qms_book_submit(self, **post):
        quid = post.get("quid")
        name = (post.get("customer_name") or "").strip()
        phone = (post.get("customer_phone") or "").strip()
        if not quid:
            return request.redirect("/qms/book?error=Please%20choose%20a%20queue")
        queue = request.env["qms.queue"].sudo().browse(int(quid)).exists()
        if not queue:
            return request.redirect("/qms/book?error=Invalid%20queue")
        Partner = request.env["res.partner"].sudo()
        partner = Partner.search([("phone", "=", phone)], limit=1) if phone else
↳ Partner
        if not partner:
            partner = Partner.create({"name": name or "Queue Customer",
                                     "phone": phone or False})
```

```
ticket = request.env["qms.ticket"].sudo().create({
  "quid": queue.id, "customer_id": partner.id,
  "ticket_name": name or partner.name})
return request.render("qms.qms_book_success", {"ticket": ticket})
```

9.1 Lessons

- `@http.route(...)` maps a URL to a Python method. Key parameters:
 - `type="http"` returns HTML/redirects (vs `type="json"` for RPC).
 - `auth="public"` means no login required (vs `"user"` = logged in).
 - `website=True` enables the website context (themes, `website.layout`).
 - `methods=...` restricts the HTTP verb.
- `request.env["model"]` is your gateway to the ORM from a controller.
- `.sudo()` runs as superuser, bypassing access rights. Needed because the public visitor is not in `group_user`. Use it deliberately, only where safe.
- `request.render("xmlid", values)` renders a server-side QWeb template with a context dictionary.
- `browse(id).exists()` — `browse` builds a recordset from an id (no query yet); `.exists()` confirms it is real (defends against tampered ids).

Chapter 10

The frontend layer: static/src/ (OWL / JS / CSS)

This is the backend web client runtime: **OWL** (Odoo's React-like framework) components, written in JS plus QWeb templates, bundled via the manifest `assets`. It powers the QMS Console (a *client action*).

10.1 qms_console.js — the OWL component

```
/** @odoo-module */                                // marks this as an Odoo ES module

import { registry } from "@web/core/registry";      // global registries
import { useService } from "@web/core/utils/hooks"; // access framework
↳ services
import { Component, useState, onWillStart } from "@odoo/owl"; // OWL primitives

export class QmsConsole extends Component { // an OWL component (like a React
↳ component)
  setup() { // setup() = the constructor hook
    this.action = useService("action"); // service to open actions (doAction)
    this.orm = useService("orm");       // service to call the ORM over RPC
    this.state = useState({ collapsed: false, stats: {} }); // reactive state
    onWillStart(() => this.loadStats()); // lifecycle: run before first render
  }

  get navItems() { // a getter -> used by the template's
↳ t-foreach
    return [
      { label: "Tickets", icon: "fa-ticket", action: "qms.action_qms_ticket"
↳ },
      { label: "Queues", icon: "fa-list-ol", action: "qms.action_qms_queue"
↳ },
      { label: "Service Plans", icon: "fa-cubes", action:
↳ "qms.action_qms_service_plan" },
```

```

        { label: "Services", icon: "fa-briefcase", action:
↪ "qms.action_qms_service" },
      ];
    }

    async loadStats() {                                     // call the ORM from JS (async RPC)
      const [waiting, serving, queues, services, plans] = await Promise.all([
        this.orm.searchCount("qms.ticket", [["state", "=", "waiting"]]),
        this.orm.searchCount("qms.ticket", [["state", "=", "serving"]]),
        this.orm.searchCount("qms.queue", []),
        this.orm.searchCount("qms.service", []),
        this.orm.searchCount("qms.service.plan", []),
      ]);
      this.state.stats = { waiting, serving, queues, services, plans }; //
↪ re-render
    }

    toggle() { this.state.collapsed = !this.state.collapsed; } // flip -> reactive
↪ update

    openAction(xmlid) { this.action.doAction(xmlid); } // open a standard view by
↪ xmlid
  }

  QmsConsole.template = "qms.QmsConsole";                 // bind component to its QWeb
↪ template
  registry.category("actions").add("qms_console", QmsConsole); // register under the
↪ tag

```

10.1.1 Lessons

- **OWL** is React for Odoo: components with `setup()`, reactive `useState`, lifecycle hooks (`onWillStart`), and a QWeb template.
- **Services** are framework singletons obtained via `useService: orm` (the ORM over RPC — same `search/searchCount` API, `async`), `action` (open actions).
- **The registry** wires things by string key. `registry.category("actions").add("qms_console", ...)` registers the component under the tag the `ir.actions.client` record referenced. That tag string is the glue between XML and JS.
- `this.action.doAction("qms.action_qms_ticket")` opens the standard view from JS (the “launcher” behaviour).
- **Reactivity**: mutating `this.state.x` re-renders automatically; no manual DOM work.

10.2 qms_console.xml — the OWL template

```

<templates xml:space="preserve">                         <!-- OWL template bundle
↪ -->
  <t t-name="qms.QmsConsole" owl="1">                   <!-- name MUST match
↪ Component.template -->

```

```

    <div class="o_qms_console d-flex">
      <div class="o_qms_sidebar border-end bg-light p-2"
        t-att-class="{ 'o_qms_collapsed': state.collapsed }"> <!--
→   dynamic class -->
        <button t-on-click="toggle"><i class="fa fa-bars"/></button> <!--
→   event -> method -->
        <ul class="nav flex-column">
          <li t-foreach="navItems" t-as="item" t-key="item.action"> <!--
→   loop + key -->
            <a href="#" t-on-click.prevent="() =>
→   this.openAction(item.action)"
              t-att-title="item.label">
                <i class="fa me-2" t-att-class="item.icon"/>
                <span t-if="!state.collapsed" t-esc="item.label"/>
→   <!-- hide when collapsed -->
              </a>
            </li>
          </ul>
        </div>
        <div class="o_qms_content flex-grow-1 p-4">
          <h1>Queue Management System</h1>
          <div class="card text-center">
            <div class="display-6" t-esc="state.stats.waiting or 0"/> <!--
→   bound to state -->
            <div class="text-muted">Waiting</div>
            </div>
            <a href="/qms/book" target="_blank">Public booking page</a>
          </div>
        </div>
      </div>
    </t>
  </templates>

```

10.2.1 Lessons

- Same QWeb language as the website templates, but this runs **client-side** (compiled to JS in the browser) — same syntax, different runtime.
- `t-on-click="toggle"` binds a DOM event to a component method; `t-on-click.prevent="() => ..."` uses an arrow function with `preventDefault`.
- `t-att-class="{ 'cls': condition }"` toggles a class reactively, driving the collapse animation.
- `t-foreach/t-key` render the nav list; `t-esc="state.stats.waiting"` binds text to reactive state — when `loadStats()` updates state, these update automatically.

10.3 qms_console.css — the collapse styling

```
.o_qms_sidebar { width: 220px; flex: 0 0 auto; transition: width .15s ease;
↪ overflow: hidden; }
.o_qms_sidebar.o_qms_collapsed { width: 56px; }    /* the toggled class shrinks it
↪ */
.o_qms_content { overflow: auto; }
```

- Plain CSS bundled into `web.assets_backend`. The JS toggles the `o_qms_collapsed` class; CSS does the smooth width animation. Separation of concerns: JS = state, CSS = presentation.

Chapter 11

The end-to-end flow

1. **Install.** Odoo reads `__manifest__.py`, imports `__init__` (registers models -> creates tables), loads `data` (security, views, menus, website templates), and bundles `assets`.
2. **Backend.** Click the app -> `menu_qms_home` -> `action_qms_console` (client action, tag `qms_console`) -> the registry finds `QmsConsole` -> OWL renders the sidebar and stats (via the `orm` service). Click a sidebar item -> `doAction` -> a standard `act_window` view, rendered by Odoo's generic renderer from your XML.
3. **Public.** A visitor hits `/qms/book` -> `QmsPublicController` -> reads queues via the ORM (`sudo`) -> renders a server-side QWeb template -> POST creates a partner and a ticket -> a success page.

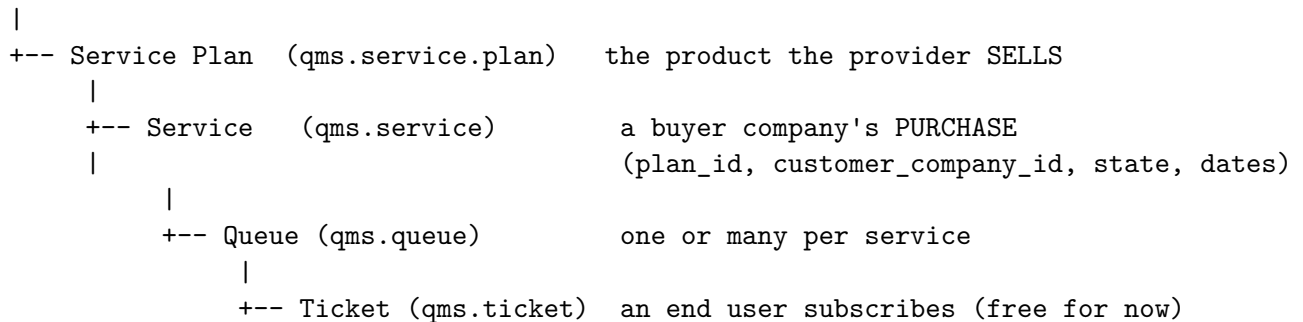
That is the full Odoo architecture in one module: **manifest** -> **ORM models** -> **security** -> **declarative views/actions/menus** -> **HTTP controllers** -> **OWL frontend**, spanning the two runtimes (backend web client and website).

Chapter 12

The data model and concept

QMS models a multi-tenant, SaaS-style queue system.

QMS Provider ("Me" = Odoo res.company)



Customer (end user) ---- customer_id ----> res.partner

Buyer Company ---- customer_company_id (is_company) ----> res.partner

- **Three tiers:** the QMS provider (the platform owner), buyer companies (who purchase a plan and run queues), and end users (who take tickets).
- A service consists of one or more queues.
- End users subscribe to a ticket for free now; paid tickets are a future layer.
- **position** and **notified** on the ticket support the core idea: notify a customer just before their turn so they avoid physical waiting.

Chapter 13

Development and deployment workflow

The QMS module is developed locally and deployed into a running Odoo container.

- **Source location:** `projectspace/myodoo/addons/qms`. The `addons` directory is bind-mounted into the container `myodoo-app`, so code edits are visible immediately — **no image rebuild** is needed.
- **Database:** `myodoo` on the container `mypostgresql_db-container`.

13.1 Common commands

Upgrade the module after changing models/views (additive changes):

```
docker exec -w /myodoo/odoo myodoo-app ./odoo-bin \  
-c /myodoo/odoo/odoo.conf -d myodoo -u qms --stop-after-init --no-http  
docker restart myodoo-app
```

Reinstall cleanly after a schema-breaking change (removing/renaming fields). First uninstall via the ORM shell, then install fresh:

```
# uninstall (via odoo shell): mod.button_immediate_uninstall()  
docker exec -w /myodoo/odoo myodoo-app ./odoo-bin \  
-c /myodoo/odoo/odoo.conf -d myodoo -i qms --stop-after-init --no-http  
docker restart myodoo-app
```

Run an ORM script (e.g. create demo data):

```
docker exec -i myodoo-app bash -lc \  
"cd /myodoo/odoo && ./odoo-bin shell -c /myodoo/odoo/odoo.conf -d myodoo  
↪ --no-http" \  
< my_script.py
```

After any backend asset change (JS/CSS/QWeb), **hard-refresh** the browser (Ctrl+Shift+R) so the new bundle loads.

Chapter 14

Glossary / cheat-sheet

- **Model** — a Python class mapped to a PostgreSQL table (`models.Model`).
- **Recordset** — a collection of records; ORM methods operate on sets.
- **Field** — a declarative column type (`Char`, `Integer`, `Many2one`, `One2many`, `Selection`, `Datetime`, ...).
- **Many2one** — a foreign key to another model.
- **One2many** — the virtual reverse side of a `Many2one`.
- **Computed field** — a field whose value comes from a method (`compute=`), recomputed per `@api.depends`.
- **Domain** — a search filter, a list of (`field`, `operator`, `value`).
- **Action** — what a menu/button opens: `act_window` (model views), `client` (OWL screen), `act_url` (a URL), `report` (PDF), `server` (Python).
- **View** — an `ir.ui.view` record describing a screen (`tree`, `form`, `search`, `kanban`, ...).
- **Widget** — how a single field is rendered (`statusbar`, `handle`, `color_picker`, ...).
- **QWeb** — Odoo's templating language; server-side (Python) for the website, client-side (JS) for OWL.
- **OWL** — Odoo's React-like frontend framework.
- **Service** — a frontend singleton (`orm`, `action`, `notification`, ...) obtained via `useService`.
- **Registry** — the global string-keyed lookup that wires components, fields, and actions.
- `sudo()` — run ORM calls as superuser, bypassing access rights.
- `xmlid` — `module.record_id`, the stable external identifier of a record.

Chapter 15

Where this fits in the learning roadmap

QMS is built as a tour of Odoo UI techniques. Progress so far:

- **Declarative model views** (list/form/search) — done (F1).
- **Public website page + controller** (/qms/book) — done (F8).
- **Client action with a collapsible sidebar** (the Console) — done.

Planned next (each demonstrates another technique): kanban board, pivot and graph analytics, a custom field widget, a wizard (TransientModel), a systray counter, a website snippet, a customer portal, a live display board (frontend JS), a QWeb PDF report, and a transactional email template.

End of book.